

GitHub Repository: https://github.com/yosammy01/CSC8830_COMPUTER_VISION/tree/main/MODULE3

Web Application: https://yosammy01.github.io/CSC8830_COMPUTER_VISION/#/module3

- I. Implement: Image blurring using a filtering approach.
- `spatial_blurring_filter.py` script uses two low-pass filters to blur an image of myself. The filters used were Box and Gaussian. The total value of all the cells of each filter, added together, is equal to 1, to preserve the brightness after the convolution step. This is standard for low-pass filters.
 - The first filter is a 5x5 Box filter that contains a value of $1/25$ in each cell. This was executed using OpenCV's `filter2D` method, for custom filters.

1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25
1/25	1/25	1/25	1/25	1/25

- The second filter is a 5x5 Gaussian filter. The cell values were automatically calculated by OpenCV's `GaussianBlur` method. The actual values of the filter were obtained with a separate program for presentation of the values here.

```
import cv2
import numpy as np

# 1. Generate the 1D kernel (size=5, sigma=0)
# This returns a 5x1 column vector
kernel_1d = cv2.getGaussianKernel(5, 0)

# 2. Create the 2D 5x5 kernel using the outer product
# (multiplying the vertical 1D kernel by a horizontal 1D kernel)
kernel_2d = np.outer(kernel_1d, kernel_1d)

# 3. Print the matrix (rounded for readability)
print("5x5 Gaussian Kernel:")
print(np.round(kernel_2d, 4))

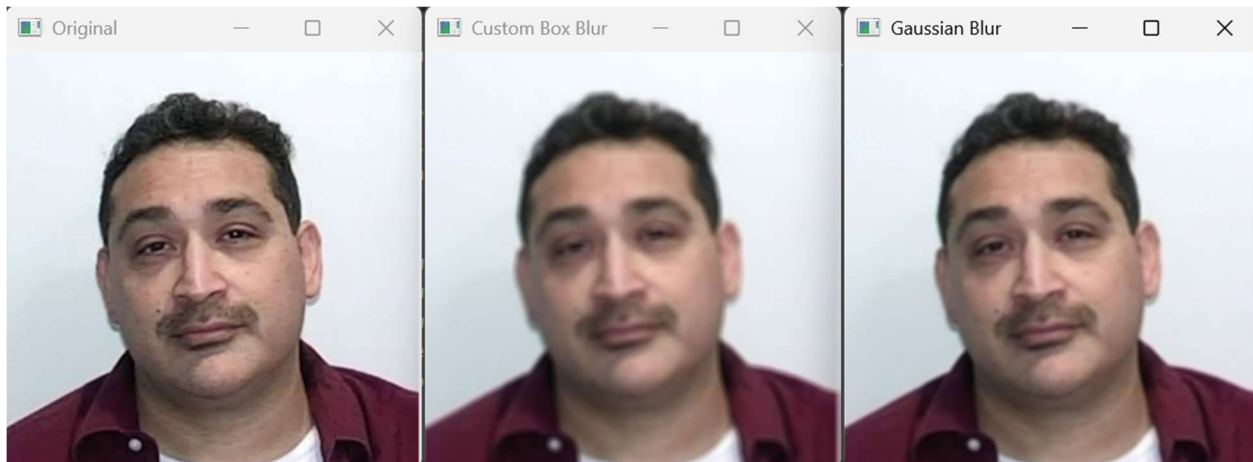
# 4. Verify the sum
kernel_sum = np.sum(kernel_2d)
print(f"\nSum of all elements: {kernel_sum:.4f}")

5x5 Gaussian Kernel:
[[0.0039 0.0156 0.0234 0.0156 0.0039]
 [0.0156 0.0625 0.0938 0.0625 0.0156]
 [0.0234 0.0938 0.1406 0.0938 0.0234]
 [0.0156 0.0625 0.0938 0.0625 0.0156]
```

$[0.0039 \ 0.0156 \ 0.0234 \ 0.0156 \ 0.0039]$

Sum of all elements: 1.0000

- d. The original and the two filtered images are then displayed to verify the blurring results:



- II. Theory: Show that the outcome using spatial filters is the same as using the fourier domain equivalent of the filter. Show that convolution in space is the same as multiplication in frequency (fourier) domain
- a. Math showing relationship between convolution and fourier transforms, with emphasis on image filtering.

Theory:

① Spatial Filtering is the process of convoluting the image with a filter (kernel).

$$g(x,y) = f(x,y) * h(x,y)$$

$\uparrow$ $\uparrow$ $\uparrow$
 filtered image image filter (kernel)

② The convolution theorem states that convolution in the spatial domain equates to multiplying the Fourier Transforms of the functions and then taking the inverse Fourier Transform of that result.

$$G(u,v) = \mathcal{F}\{g(x,y)\}$$

$$F(u,v) = \mathcal{F}\{f(x,y)\}$$

$$H(u,v) = \mathcal{F}\{h(x,y)\}$$

$$G(u,v) = F(u,v) \cdot H(u,v)$$

$$g(x,y) = \mathcal{F}^{-1}\{G(u,v)\}$$

③ So, the filtered image can be obtained by taking the inverse Fourier Transform of the result of multiplying the Fourier Transforms of the image and the filter (kernel).

- b. Use the implementation from above and experimentation for showing evidence/validation.
- i. spatial_vs_frequency_blurring_filter.py uses the same image and spatial Box filter from the spatial_blurring_filter.py script. It also uses the Fourier Transforms of the same image and Box Filter. The original image is converted to gray scale to simplify the Fourier transform math.
 - ii. The Box filter is used to filter the image as before, as the filtered, control image.
 - iii. The Box filter is then placed in the top left corner of a zero-filled matrix with the same pixel dimensions as the image. Then the Box filter is positioned so its center cell is in the top left corner or origin of the pixel matrix (0,0). This pushes the filter to all corners of the pixel matrix. These are requirements of the Fast Fourier Transform that is used.

1	2	3	0	0	0	...
4	5	6	0	0	0	...
7	8	9	0	0	0	...

0	0	0	0	0	0	...
0	0	0	0	0	0	...
0	0	0	0	0	0	...
...	...	...	...	...	...	...

5	6	0	0	0	...	4
8	9	0	0	0	...	7
0	0	0	0	0	...	0
0	0	0	0	0	...	0
0	0	0	0	0	...	0
...	...	...	...	...	...	...
2	3	0	0	0	...	1

- iv. The Fourier Transforms of the image and the Box filter are then created and they are multiplied with each other. The Inverse Fourier Transform is then used on the that result.
- v. The spatially-filtered and Fourier-Transform-processed images are then displayed while the filtered image and Fourier-Transform-processed images are subtracted from each other – not counting the borders because there are differences along the borders caused by the two different ways borders are handled by OpenCV's filter2D and FFT methods – and that resulting image is also shown (should be black to show exact match). The maximum error between the two resulting images is also computed.

Maximum difference between spatial and frequency methods: 0.0002

